

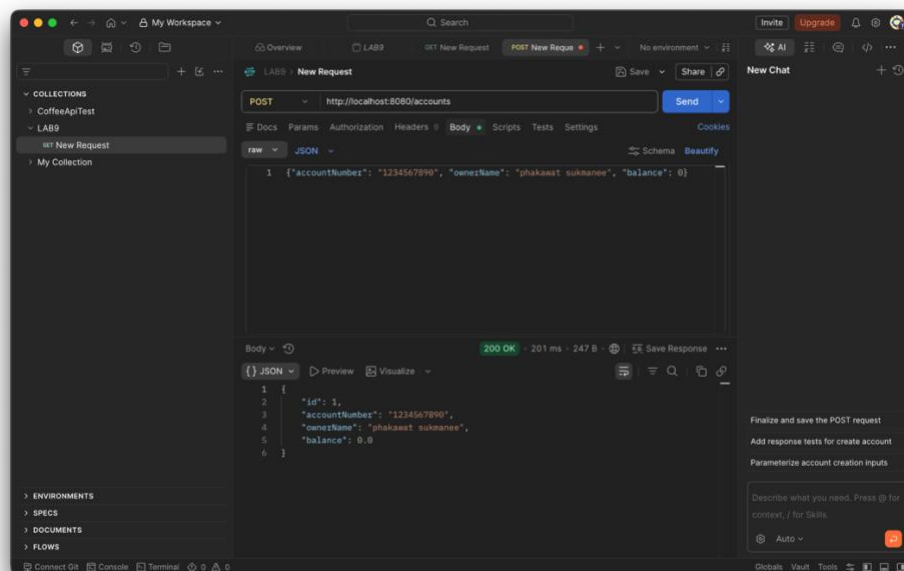
รายงานผลการทดลอง Lab 9: Spring Boot – Transaction

ชื่อ-นามสกุล: ภาควัฒน์ สุขมณี

รหัสนักศึกษา: 673380418-9

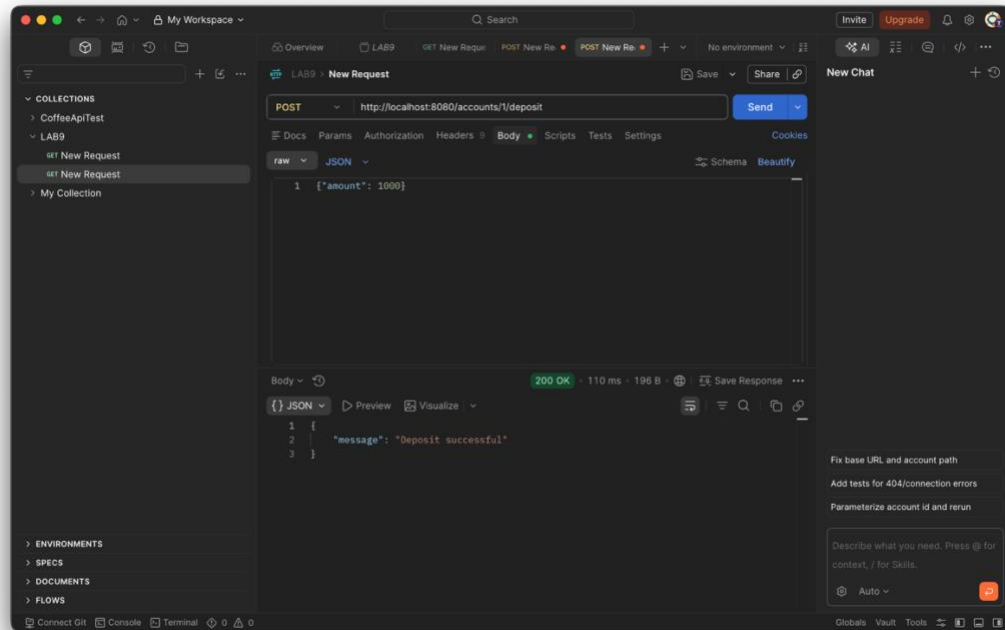
Section: 4

1. การสร้าง Account



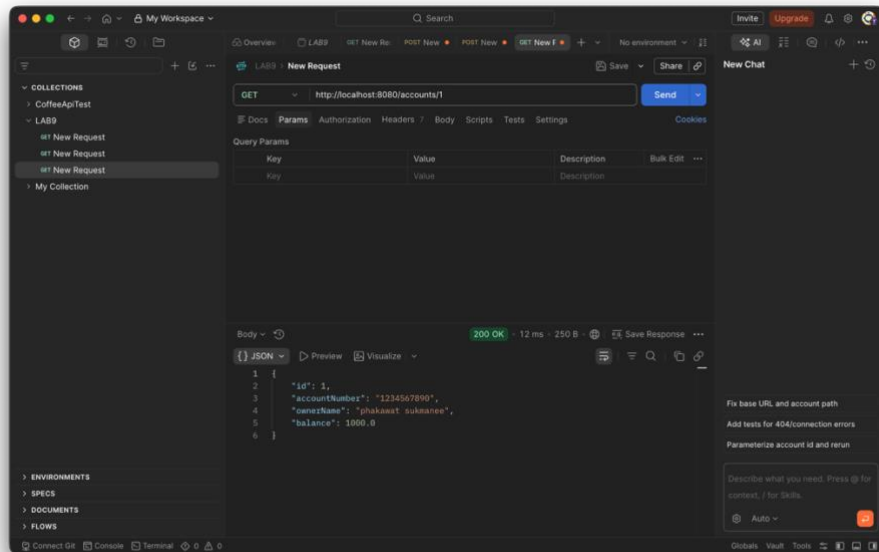
อธิบายผล : จากการยิง Request POST /accounts พร้อมข้อมูล accountNumber, ownerName และ balance เริ่มต้นเป็น 0 ระบบสามารถสร้างบัญชีใหม่ได้สำเร็จ โดย Response กลับมาเป็น Status 200 OK พร้อม id ของบัญชีที่ถูกสร้างขึ้นในฐานข้อมูล ซึ่งจะนำ id นี้ไปใช้อ้างอิงในการทดสอบขั้นต่อไป

2. การฝากเงินสำเร็จ



อธิบายผล : เมื่อยิง Request POST /accounts/{id}/deposit พร้อมจำนวนเงินที่ต้องการฝาก ระบบทำงาน 2 ขั้นตอนภายใน Transaction เดียวกัน คือ (1) เพิ่มยอดเงินใน Account และ (2) บันทึกประวัติการฝากลงตาราง DepositTransaction ผลลัพธ์คือ Response กลับมาเป็น 200 OK พร้อมข้อความ "Deposit successful" แสดงว่าทั้งสองขั้นตอนสำเร็จพร้อมกัน

3. ตรวจสอบ Account



อธิบายผล : เมื่อยิง GET /accounts/{id} เพื่อตรวจสอบยอดเงิน พบว่าค่า balance เปลี่ยนแปลงตรงตามจำนวนเงินที่ฝากเข้าไปจริง ยืนยันว่าขั้นตอนการเพิ่ม balance ในเมธอด deposit() ทำงานถูกต้อง

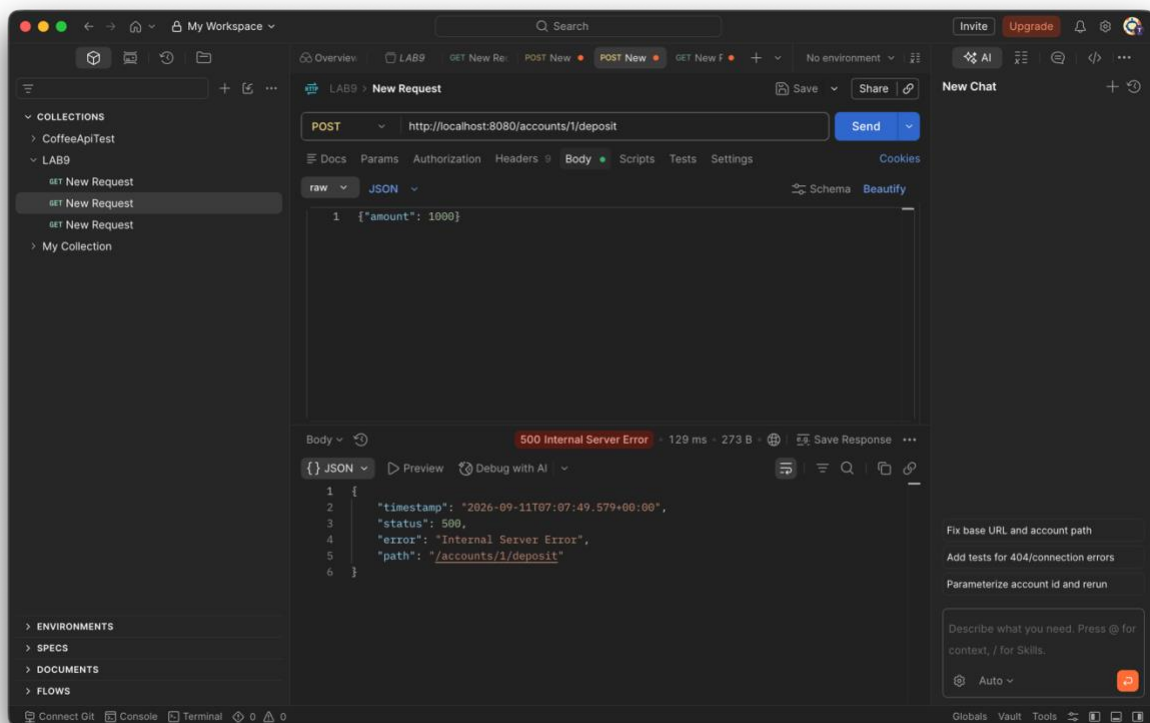
4. ตรวจสอบ DepositTransaction

```
lab9=# SELECT * FROM deposit_transaction;  
 id | amount | account_id  
-----+-----+-----  
  1 |   1000 |          1  
(1 row)
```

อธิบายผล : จากคำสั่ง SELECT * FROM deposit_transaction; ใน Terminal พบว่ามีแถวข้อมูลที่มี amount ตรงกับจำนวนเงินที่ฝาก และ account_id อ้างอิงกลับไปยัง Account ที่ถูกต้อง แสดงว่าความสัมพันธ์แบบ Many-to-One ระหว่าง DepositTransaction กับ Account ทำงานตามที่ออกแบบไว้

5. ทดลอง Rollback (มี @Transactional)

```
lab9=# SELECT * FROM deposit_transaction;
 id | amount | account_id
-----+-----+-----
  1 |   1000 |         1
(1 row)
```



อธิบายผล : หลังจากเพิ่มบรรทัด throw new RuntimeException("Test Rollback"); ไว้ท้ายเมธอด deposit() (หลังจาก save ข้อมูลไปแล้วทั้งสองขั้นตอน) แล้วยิง Request เข้า ระบบตอบกลับเป็น 500 Internal Server Error ตามที่คาดไว้ เมื่อตรวจสอบยอดเงินและตาราง deposit_transaction พบว่า ไม่มีการเปลี่ยนแปลงใดๆ เกิดขึ้นจริง ทั้งที่โค้ดสั่ง save() ไปแล้วก่อนเกิด Error แสดงว่า @Transactional สั่ง ROLLBACK ยกเลิกการเปลี่ยนแปลงทั้งหมดในก้อนเดียวกันสำเร็จ

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

```
[lab9=# SELECT * FROM deposit_transaction;  
 id | amount | account_id  
-----+-----+-----  
  1 |   1000 |          1  
  2 |   1000 |          1  
  3 |   1000 |          1  
  5 |   1000 |          1  
  6 |   1000 |          1  
(5 rows)
```

อธิบายผล : เมื่อลบ @Transactional ออกจากเมธอด deposit() โดยยังคงบรรทัด throw new RuntimeException("Test Rollback"); ไว้เหมือนเดิม แล้วทดลองฝากเงินซ้ำ ผลลัพธ์คือระบบยังคงตอบกลับ 500 Internal Server Error เหมือนเดิม แต่เมื่อตรวจสอบฐานข้อมูลพบว่า balance เพิ่มขึ้นจริง และมีแถวใหม่ถูกบันทึกลงตาราง deposit_transaction จริง ทั้งที่ Request นั้น Error จะเห็นได้ว่าเมื่อไม่มี @Transactional คำสั่ง save() แต่ละคำสั่งจะถูกบันทึกลงฐานข้อมูลทันทีที่คำสั่ง แยกจากกัน ไม่มีกลไกใดมองภาพรวมของงานทั้งก้อน เมื่อเกิด Error ขึ้นภายหลัง จึงไม่มีการย้อนกลับข้อมูลที่บันทึกไปแล้วให้

ตอบคำถามท้ายแลป

7. ตอบคำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ANS : ทำหน้าที่กำหนดขอบเขตของ Transaction โดยรวมคำสั่งทั้งหมดภายในเมธอดให้นับเป็นงานก้อนเดียวกัน ถ้าทุกคำสั่งสำเร็จจะ COMMIT ข้อมูลทั้งหมดพร้อมกัน แต่ถ้ามีคำสั่งใดคำสั่งหนึ่งเกิด Error จะ ROLLBACK ยกเลิกการเปลี่ยนแปลงทั้งหมดในก้อนนั้น

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ANS : เพราะการฝากเงิน 1 ครั้งมีการเปลี่ยนแปลงข้อมูลมากกว่า 1 อย่างพร้อมกัน คือเพิ่ม balance ใน Account และบันทึกประวัติลง DepositTransaction ทั้งสองขั้นตอนนี้ควรสำเร็จไปด้วยกันเสมอ หากขั้นตอนใดขั้นตอนหนึ่งล้มเหลว ข้อมูลทั้งหมดของการฝากครั้งนั้นต้องถูกยกเลิกไปด้วย เพื่อไม่ให้ข้อมูลไม่สอดคล้องกัน (เช่น บันทึกประวัติไปแล้วแต่ยอดเงินไม่เพิ่ม)

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ANS : COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน Transaction นั้นลงฐานข้อมูลอย่างถาวร เมื่อทุกคำสั่งทำงานสำเร็จ ส่วน ROLLBACK คือการยกเลิกและย้อนคืนการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน Transaction นั้นกลับไปเหมือนก่อนเริ่ม เมื่อมีคำสั่งใดคำสั่งหนึ่งเกิด Error ระหว่างทาง

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ANS : เป็นความสัมพันธ์แบบ One-to-Many จากฝั่ง Account ไปยัง DepositTransaction กล่าวคือ 1 Account สามารถมี DepositTransaction ได้หลายรายการ โดยฝั่ง DepositTransaction เป็นผู้เก็บ Foreign Key account_id ที่อ้างอิงกลับไปยัง Account (Many-to-One)

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ANS : ข้อมูลจะไม่มีการเปลี่ยนแปลงใดๆ เกิดขึ้นจริงในฐานข้อมูล แม้ว่าโค้ดจะสั่ง save() ไปแล้วก่อนเกิด Error ก็ตาม เพราะ @Transactional จะสั่ง ROLLBACK ยกเลิกการเปลี่ยนแปลงทั้งหมดในก้อนนั้นกลับไปเหมือนก่อนเริ่ม Transaction

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ANS : เมื่อลบ @Transactional ออก แม้จะยังเกิด Error เหมือนเดิม แต่ข้อมูลที่ถูก save() ไปก่อนหน้าจุดที่ Error เกิดขึ้น จะยังคงถูกบันทึกอยู่ในฐานข้อมูลจริง ไม่มีการย้อนกลับ เนื่องจากไม่มีกลไกใดมองภาพรวมของงานทั้งก้อนอีกต่อไป คำสั่ง save() แต่ละคำสั่งจะจบการทำงานแยกจากกันทันที